

Answer Key: ArrayList Review Handouts

Handout 1: Warm-Up (What's in the List?)

#	Answer
1	[4, 10, 19, 20, 4]
2	words = [yellow, blue, green, purple, blue]; old = "red"
3	list = [20, 15, 10, 25, 5]; a = 15
4	items = [E, D, C, B]; x = "B"; y = "D"
5	[2, 10, 6, 16]
6	[3, 6, 9]
7	[2, 1, 0, 2, 4, 6, 8, 10, 12] (each add(0, i) pushes the earlier ones right, so the new values appear in reverse)
8	[8, 10, 12, 3, 5, 7]
9	[2, 4]
10	[2, 4, 8, 12]

Notes - Problem 2: `set` returns the old value, so the inner `words.set(2, "purple")` returns "blue", and that is what gets stored at index 3. - **Problem 4:** `set(2, x)` returns "D", which is y. The last line removes "E" from the end and stores it at index 0. - **Problem 5:** `data.get(0) - 1` is an `int` (`2 - 1 = 1`), so this is `remove(1)`, which takes an index. - **Problems 9 and 10:** Same condition, opposite directions. Going forward, each `remove(i)` slides the next element into index i, then `i++` skips it. In Problem 10, 8 and 12 are never tested, and 10 is only tested after 8 has slid down.

Handout 2: Trace These

Problem 4: Prints

Alex Bob Carl
Alex Alex Alex

`set` returns the old value, so the first loop prints the original names while overwriting each with "Alex".

k	students before	returned by <code>set</code>	students after
0	[Alex, Bob, Carl]	Alex	[Alex, Bob, Carl]
1	[Alex, Bob, Carl]	Bob	[Alex, Alex, Carl]
2	[Alex, Alex, Carl]	Carl	[Alex, Alex, Alex]

Problem 7: `numList` ends as `[2, 20, 16]` and the method returns `[5, 10]`. The intended results were `[2, 16]` and `[5, 10, 20]`.

i	size	num	divisible?	numList after	returnList after
0	5	5	yes	<code>[2, 10, 20, 16]</code>	<code>[5]</code>
1	4	10	yes	<code>[2, 20, 16]</code>	<code>[5, 10]</code>
2	3	16	no	<code>[2, 20, 16]</code>	<code>[5, 10]</code>

The loop then ends (`i = 3`, size 3). It first goes wrong at `i = 1`: after 10 is removed, 20 slides into index 1, but `i++` moves on to index 2, so 20 is never checked.

Problem 8: `values` ends as `[0, 4, 2, 5, 3]`.

k	size	<code>nums.get(k)</code>	removed?	nums after
0	8	0	yes	<code>[0, 4, 2, 5, 0, 3, 0]</code>
1	7	4	no	same
2	7	2	no	same
3	7	5	no	same
4	7	0	yes	<code>[0, 4, 2, 5, 3, 0]</code>
5	6	0	yes	<code>[0, 4, 2, 5, 3]</code>

The loop then ends (`k = 6`, size 5). The method does not remove every 0. After the first removal, the second 0 slid into index 0 and was never examined, so a 0 survives at the front.

Problem 9: `myData` ends as `[3, 4, 4, 8, 7, 7]`, not `[3, 4, 8, 7]`.

k	<code>get(k)</code>	<code>get(k - 1)</code>	equal?	myData after
1	3	3	yes	<code>[3, 4, 4, 4, 8, 7, 7, 7]</code>
2	4	4	yes	<code>[3, 4, 4, 8, 7, 7, 7]</code>
3	8	4	no	same
4	7	8	no	same
5	7	7	yes	<code>[3, 4, 4, 8, 7, 7]</code>

The loop then ends (`k = 6`, size 6). After a removal, the next element slides into index `k`, but `k++` skips past it. The fix is to put an **else** before `k++`, so `k` only advances when nothing was removed.

Problem 11: Prints `[200, 400]`. The code compiles (raw `new ArrayList()` only produces a warning).

After statement	oldList	newList
oldList.add(400)	[100, 200, 300, 400]	[]
newList.add(oldList.remove(1))	[100, 300, 400]	[200]
newList.add(oldList.get(2))	[100, 300, 400]	[200, 400]

remove(1) returns 200 and shifts the rest left, so get(2) is now 400, not 300.

Handout 3: FRQ averageWithinRange

```
public double averageWithinRange(double lower, double upper)
{
    double sum = 0.0;
    int count = 0;
    for (ItemInfo item : inventory)
    {
        double cost = item.getCost();
        if (item.isAvailable() && cost >= lower && cost <= upper)
        {
            sum += cost;
            count++;
        }
    }
    return sum / count;
}
```

What to look for - Traverses every element of `inventory`. - Tests availability and both bounds, inclusive (`>=` and `<=`). - Accumulates a sum and a count of only the qualifying items. - Returns sum divided by count, as a `double`.

Common mistakes - Using `>` or `<`, which excludes the endpoints. - Dividing by `inventory.size()` instead of the count of qualifying items. - Declaring `sum` as an `int`, which loses the cents. - Forgetting the `isAvailable()` check (the watch is the test case for this). - Guarding against `count == 0`, which the precondition makes unnecessary but is harmless.